

MARS Project



*Metric-driven Automated Representation Search
for Improved Classification*



Project Aim

To automatically discover and evolve feature extraction representations that maximise classification performance on the FEI dataset using Large Language Models.



Dataset

Field Emission Images (FEI)
of emission sites



Methodology

Evolution of feature
extraction functions using
the EOH framework



Evaluation

Support Vector Machine (SVM)
classification accuracy used
as the fitness metric



Goal

Improve classification
accuracy through automated
representation search



UNIVERSITY OF
LIVERPOOL

James Davies
MSc Data Science

July 2025

Original GP vs. Modified GP

Experimental Comparison Across 10 Independent Runs

Performance Summary (10 Runs)

Metric	Original GP	Modified GP
 Best Accuracy	100.00%	100.00%
 Mean Accuracy	97.07%	95.85%
 Mean Training Time	98.87 s	95.89 s
 Mean Feature Size	655	511

Key Observations

- ✓ Both implementations achieved **100%** best classification accuracy.
- ✓ Modified GP evolved more **compact** feature representations.
- ✓ Performance remained **comparable** across ten runs.



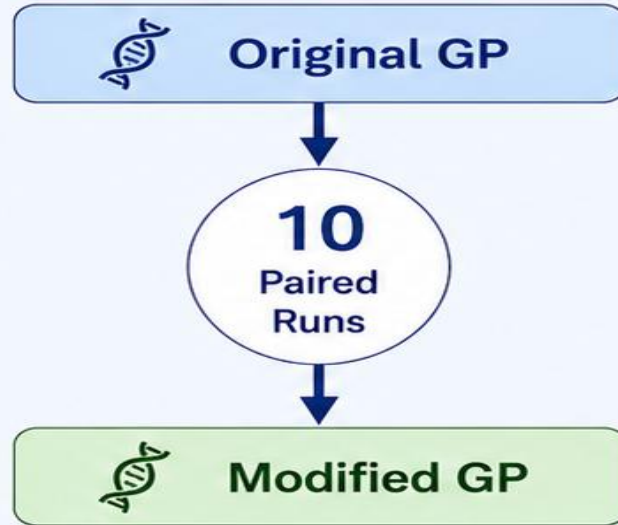
The modified GP maintains competitive classification performance while evolving smaller feature representations and slightly reducing training time.



Statistical Evaluation

Paired Comparison Across 10 Runs

Paired Experimental Design



*Same dataset, same protocol,
different implementations*

Statistical Test Results

Test	Result
 Paired t-test	$p = 0.591$ ($p > 0.05$)
 Wilcoxon signed-rank test	$p = 0.781$ ($p > 0.05$)
 Effect Size (Cohen's d)	-0.176 (Negligible)

Normality Check

Shapiro-Wilk test: $W = 0.933$, $p = 0.479$ ($p > 0.05$)

Differences are approximately normally distributed.



No statistically significant difference in classification accuracy was observed between the original and modified GP implementations.



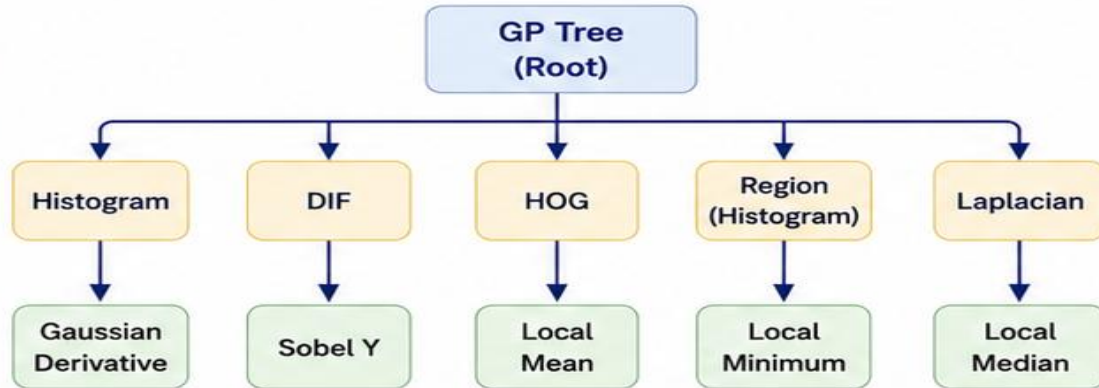
Best GP Trees (Accuracy = 100%)

Illustrating the Evolved Feature Extraction Programs

Original GP (Seed 3)



Best Accuracy: 100.00%



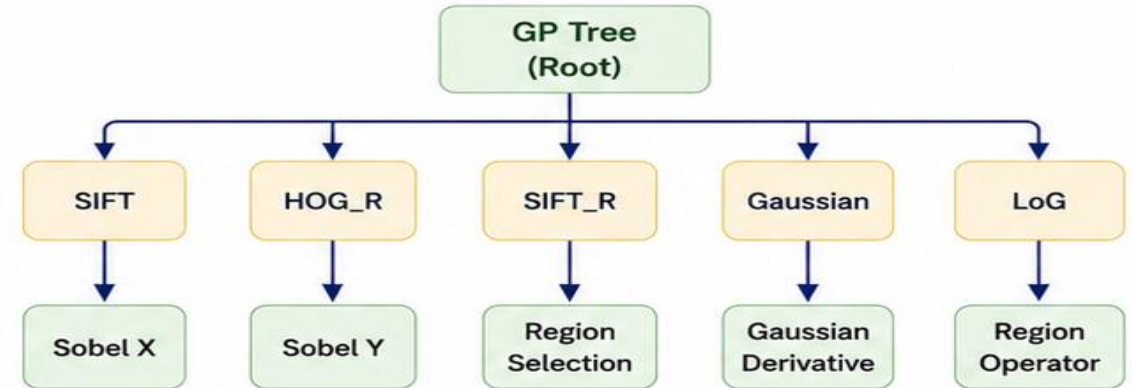
Key Characteristics

- Combines global and regional texture information
- Uses classical gradient and intensity operators
- Emphasises local statistical summaries

Modified GP (Seed 5)



Best Accuracy: 100.00%



Key Characteristics

- Utilises rotation-aware descriptors (SIFT, HOG_R, SIFT_R)
- Applies advanced filtering (Gaussian, LoG)
- Incorporates regional selection and operators



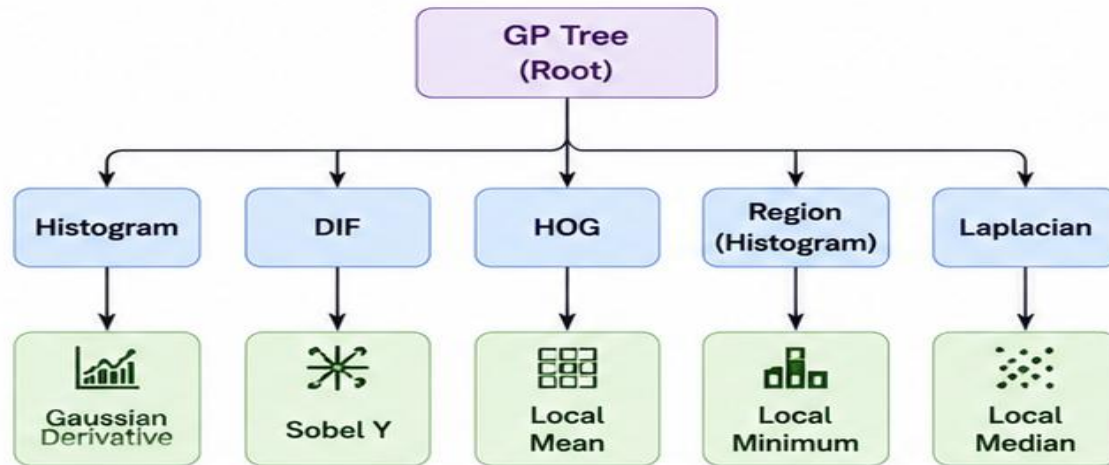
Both trees achieve perfect classification accuracy using different combinations of image processing operators and regional information.



Best Original GP Tree (Seed 3)

Achieved 100% Classification Accuracy

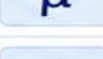
GP Tree Structure



Tree Characteristics

-  Combines global statistics with local regional information
-  Uses classical gradient and edge operators (Laplacian, Sobel)
-  Employs statistical summaries (mean, min, median) and histograms
-  Focuses on intensity distributions and local texture

Operators Used

	Histogram	Captures global intensity distribution of the image.
	DIF	Image difference operator highlighting intensity changes.
	HOG	Histogram of Oriented Gradients capturing gradient orientation distribution.
	Region (Histogram)	Histogram computed over selected image regions.
	Laplacian	Second-order derivative for edge and texture enhancement.
	Gaussian Derivative	Derivative of Gaussian for smoothing and edge detection.
	Sobel Y	Vertical edge detection using Sobel operator.
	Local Mean	Average intensity within a local neighbourhood.
	Local Minimum	Minimum intensity within a local neighbourhood.
	Local Median	Median intensity within a local neighbourhood.



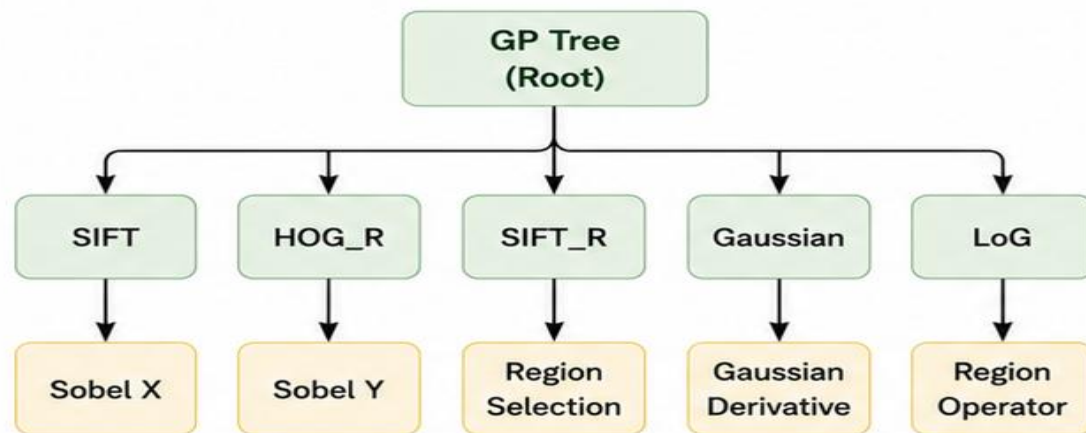
The original GP tree integrates **classical image operators** and **statistical measures** to characterise both global and local texture patterns.



Best Modified GP Tree (Seed 5)

Achieved 100% Classification Accuracy

GP Tree Structure



Tree Characteristics

- Uses rotation-aware descriptors (SIFT, HOG_R, SIFT_R)
- Applies advanced filtering (Gaussian, LoG)
- Incorporates regional selection and operators
- Combines edge detectors (Sobel X, Sobel Y) with descriptors and filters
- Focuses on robust, discriminative and rotation-invariant features

Operators Used

	SIFT	Scale-Invariant Feature Transform extractor for robust keypoint features.
	HOG_R	Rotation-aware Histogram of Oriented Gradients capturing gradient structure.
	SIFT_R	Rotation-aware SIFT variant for improved invariance.
	Gaussian	Gaussian smoothing filter to reduce noise.
	LoG	Laplacian of Gaussian for blob/edge detection at multiple scales.
	Sobel X	Horizontal edge detection using Sobel operator.
	Sobel Y	Vertical edge detection using Sobel operator.
	Region Selection	Selects informative regions of the image.
	Region Operator	Computes statistical descriptors over selected regions.



The modified GP tree leverages **rotation-aware** descriptors and **regional operators** to build robust, discriminative feature representations.



Explainability of GP Trees

What Did Evolution Learn?

Original GP

Key Operators Evolved



Histogram

Global intensity distribution



Gradient Operators (HOG, Sobel)

Edge orientation and strength



Gaussian Derivatives

Smoothing and edge detection



Regional Statistics

Mean, min, median over regions



Focus:

Classical image operators and statistical summaries

Modified GP

Key Operators Evolved



SIFT

Scale-invariant keypoints



Rotation-aware Gradients (HOG_R)

Rotation-invariant orientation info



Rotation-aware SIFT (SIFT_R)

Improved invariance to rotation



Regional & Multi-scale Operators

Selection, filtering and aggregation



Advanced Filters (Gaussian, LoG)

Noise reduction and multi-scale edges



Focus:

Rotation-aware descriptors and regional information

Key Takeaways



Evolution successfully discovered **meaningful combinations** of image processing operators.



The individual operators are **interpretable** and well understood.



However, the overall GP tree structure is **deep and nested**, making the full program difficult to interpret.



This motivates the search for more **interpretable** program representations.



Can we evolve feature extraction programs that are easier to understand?



While GP evolution identifies powerful combinations of image operators, the complexity of the resulting program trees limits their **human interpretability** and **transparency**.



Large Language Models (LLMs)

Using LLMs to Understand and Interpret Evolved GP Trees

What are LLMs?

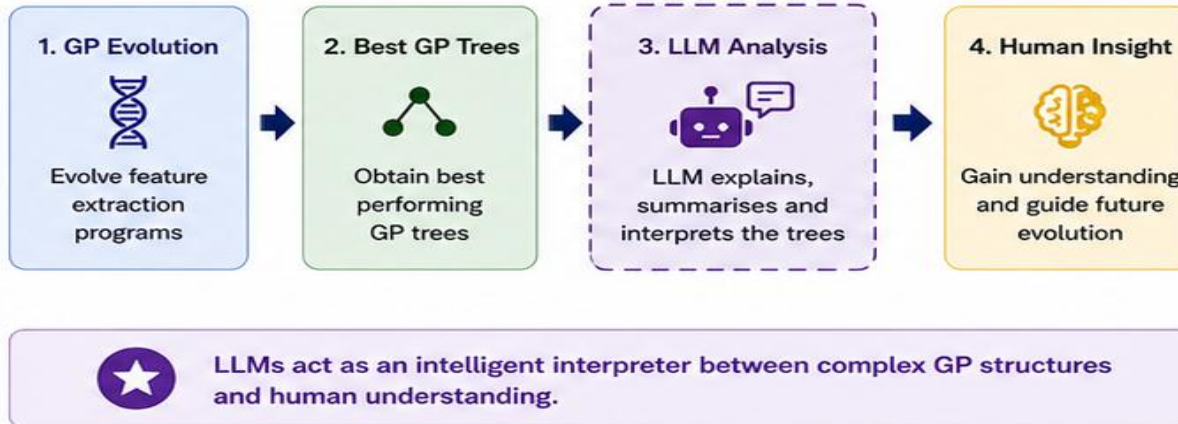


Large Language Models are **deep learning models** pre-trained on vast amounts of text data. They can understand, generate and reason about human language.

Key Capabilities

- Natural language understanding
- Reasoning and explanation
- Summarisation and comparison
- Code and technical interpretation
- Interactive question answering

Where LLMs Fit in the MARS Pipeline



Why Use LLMs?



Interpret complex GP trees in natural language.



Summarise key operations and their roles.



Compare different GP trees and operator combinations.



Identify patterns and design principles.



Guide future evolution towards more interpretable solutions.

Example Prompt to LLM



"You are an expert in image processing and genetic programming. Given the following GP tree structure used for feature extraction:

- Root: GP Tree
- Children: SIFT, HOG_R, Gaussian, LoG
- SIFT -> Sobel X
- HOG_R -> Sobel Y
- Gaussian -> Gaussian Derivative
- LoG -> Region Operator

Please explain the purpose of this program, the role of each operator, and why this combination might be effective for image classification."

Example LLM Response (Abbreviated)



- This program extracts rich, multi-scale, rotation-aware features.
- SIFT captures scale-invariant keypoints and local structures.
- HOG_R provides rotation-aware gradient orientation information.
- Gaussian smoothing and derivatives reduce noise and highlight edges.
- LoG with region operators captures blob-like structures at multiple scales.
- The combination is robust, discriminative, and well-suited for texture classification tasks.



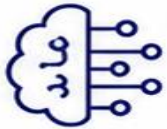
LLMs enable us to bridge the gap between complex evolved programs and human interpretability, accelerating **understanding**, **insight** and future **innovation** in the MARS project.



Large Language Models (LLMs)

Using LLMs to Understand and Interpret Evolved GP Trees

What are LLMs?



Large Language Models are deep learning models pre-trained on vast amounts of text data.

They can understand, generate and reason about human language.

Key Capabilities

- Natural language understanding
- Reasoning and explanation
- Summarisation and comparison
- Code and technical interpretation
- Interactive question answering

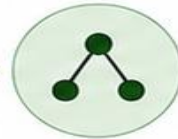
Where LLMs Fit in the MARS Pipeline



1.

GP Evolution

Evolve feature extraction programs



2.

Best GP Trees

Obtain best performing GP trees



3.

LLM Analysis

LLM explains, summarises and interprets the trees



4.

Human Insight

Gain understanding and guide future evolution



LLMs act as an intelligent interpreter between complex GP structures and human understanding.

Why Use LLMs?



Interpret complex GP trees in natural language.



Summarise key operations and their roles.



Compare different GP trees and operator combinations.



Identify patterns and design principles.



Guide future evolution towards more interpretable solutions.



LLMs help bridge the gap between complex evolved programs and human interpretability, accelerating **understanding** and driving future **innovation** in the MARS project.



Why Move Beyond Traditional GP?

Limitations of Current Evolutionary Search

Traditional GP	VS	Future with LLMs
		
 Slow to discover high-performing programs		 Faster discovery of promising programs
 Limited ability to explore complex operator combinations		 Can reason about complex combinations and patterns
 Relies on random search and mutation		 Guided search using knowledge and insight

Key Motivation



Improve efficiency of program search.



Discover more meaningful operator combinations.



Leverage LLM reasoning to guide evolution.



Aim for better interpretability without sacrificing accuracy.

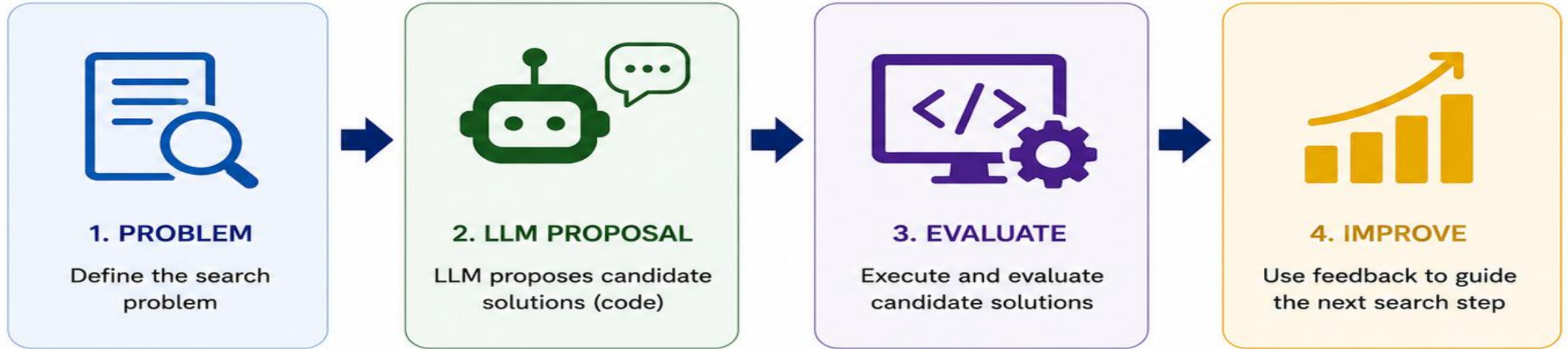


LLMs offer the potential to overcome key limitations of traditional GP and enable more **intelligent, guided** program evolution.



FunSearch

LLM-Powered Heuristic Search for Program Synthesis

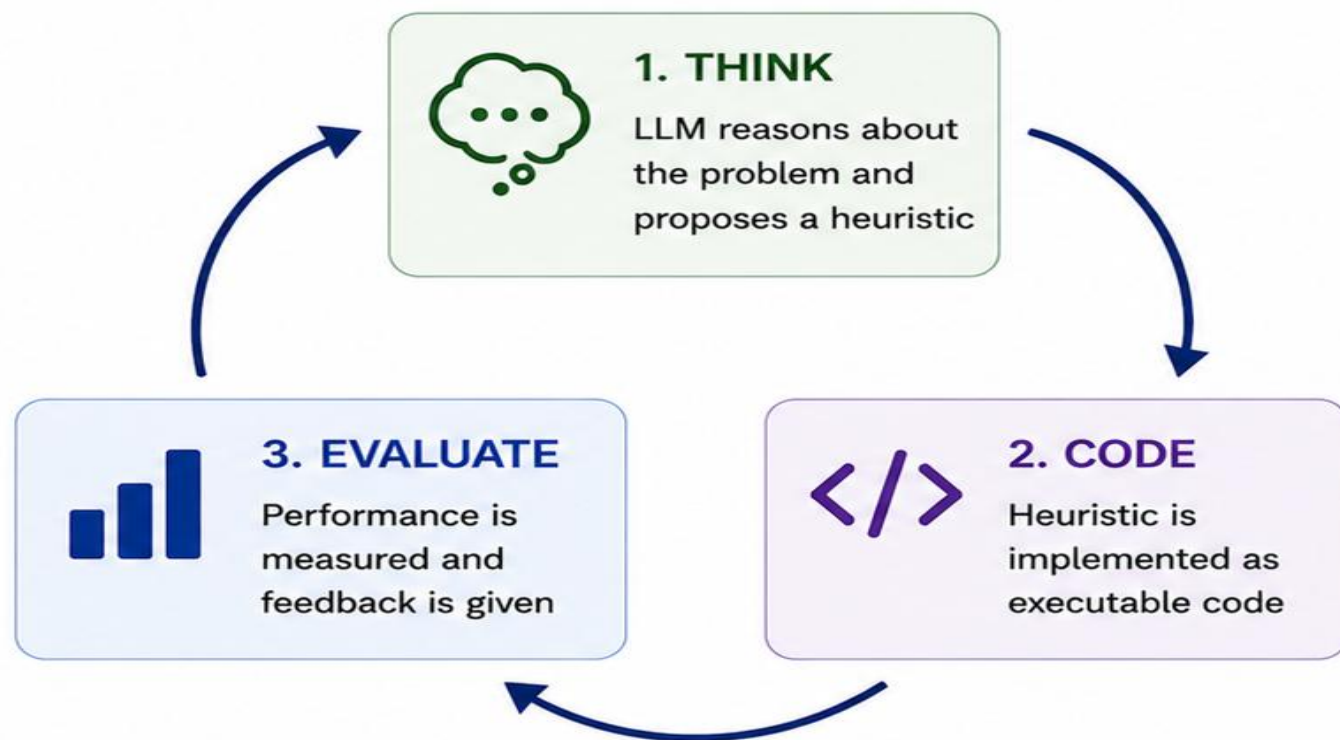


FunSearch integrates **LLM reasoning** with **heuristic search** to efficiently discover high-quality programs.



Evolution of Heuristics (EOH)

LLMs Evolve Better Search Heuristics



Key Ideas



LLMs generate heuristics informed by reasoning and prior knowledge.



Heuristics are implemented and tested in the environment.



Feedback drives the evolution towards higher-performing heuristics.



Iterative loop improves search efficiency and solution quality.

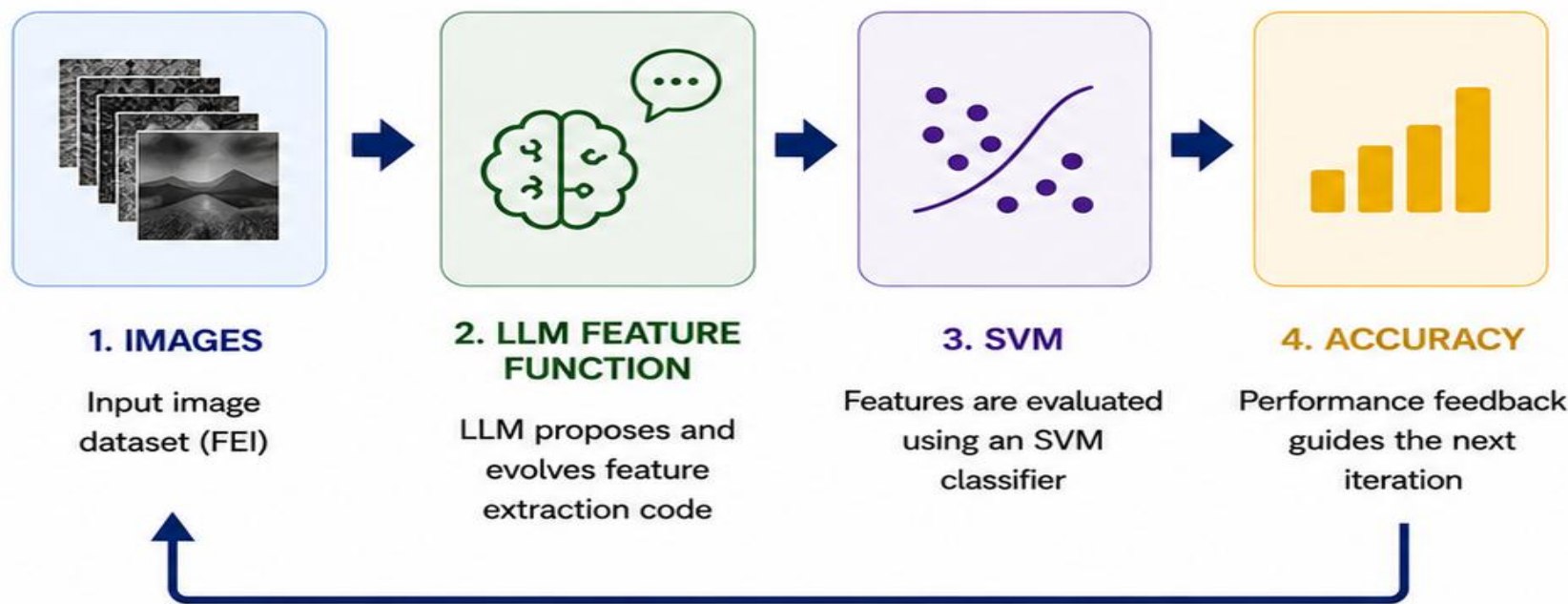


EOH enables LLMs to iteratively **design, test, and refine** heuristics, leading to more effective search and better solutions.



Applying EOH to Feature Extraction

LLMs Evolve Feature Functions for Image Classification



Key Points



LLMs generate creative feature extraction programs.



Programs are executed and evaluated.



Accuracy feedback drives the evolution.



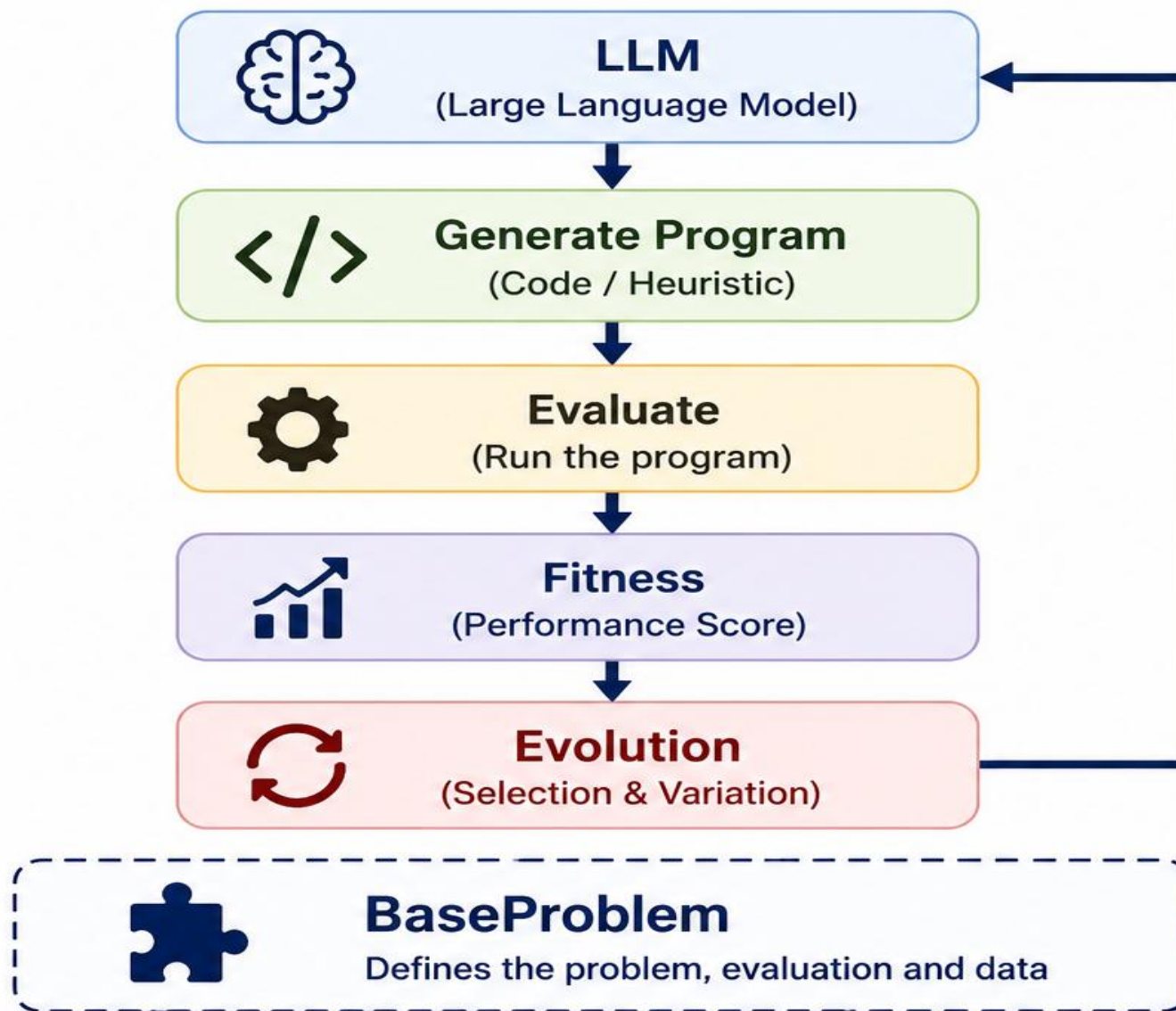
Iterative process improves feature functions over time.



EOH with LLMs enables the **automated discovery** of effective feature extraction programs for improved classification performance.



Understanding the EOH Architecture



Mapping EOH to My Project

